ARTIFICIAL WORLD DOCUMENTO DEFINITIVO PREMIUM · 2026

Artificial World: Arquitectura de un Sistema de Simulacion Multiagente Trazable y Reproducible para Civilizaciones Emergentes

Version editorial elevada del documento largo: misma ambicion conceptual, mejor jerarquia visual, maquetacion mas solemne y una presentacion a la altura de un texto fundacional serio.

> *"No persigas una apariencia de inteligencia. Construye un sistema que pueda rendir cuentas de lo que afirma."*

DOCUMENTO DEFINITIVO | PREPRINT TECNICO PREMIUM | BASELINE REPRODUCIBLE | ARQUITECTURA VERIFICABLE

AUTOR
Cosigein SL

BASE TEXTUAL
`docs/PAPER_FINAL.md`

SALIDA
Documento largo, no resumido

ABSTRACT DE PORTADA

Artificial World presenta una arquitectura multiagente trazable, reproducible y defendible.

La baseline experimental se presenta como linea base verificable para futuras comparaciones controladas.

CLAVE EDITORIAL

El documento no intenta aparentar mas ciencia de la que hoy puede defender, pero si elevar al maximo la fuerza de su arquitectura, su trazabilidad y su propuesta metodologica.

MOTOR BASE	SESION CANONICA	AGENTES	TICKS	MARCO
Python + SQLite	Seed 42	8	200	Preprint tecnico

LECTURA CORRECTA

Documento definitivo con elevacion editorial

INDICE NAVEGABLE

Mapa del documento

Abstract

1. Introduccion

TESIS CENTRAL

El valor del sistema no esta solo en lo que simula, sino en que obliga a que cada claim pueda rastrearse hasta evidencia verificable.

BASE EXPERIMENTAL

La sesion canonica es una baseline reproducible, no una validacion estadistica cerrada. Su fuerza esta en que puede repetirse y contrastarse.

POSICIONAMIENTO CORRECTO

Este documento debe leerse como pieza fundacional de alto nivel: ambiciosa, tecnica y defendible, sin vender mas evidencia de la que hoy existe.

1.1 Alcance del documento

2. Contexto y trabajo relacionado

3. Arquitectura del sistema

3.1 Motor principal en Python

3.2 Capa web separada

3.3 Sistema Chess

4. Decision por utilidad y memoria

4.1 Memoria espacial y social

4.2 Override de autonomia

5. Baseline experimental reproducible

5.1 Resultados agregados

5.2 Observaciones iniciales

5.3 Reproducibilidad

5.4 Interpretacion prudente

6. Trazabilidad radical y taxonomia de evidencia

6.1 Taxonomia del estado de los componentes

7. Modelo conceptual y flujo fundador

7.1 Regla 2D / 3D

8. Limitaciones

9. Trabajo futuro

10. Conclusiones

Referencias

Anexo minimo: reproduccion y verificacion

Reproduccion de la baseline

Artefactos esperados

Taxonomia resumida

BLOQUES PRINCIPALES

Estructura larga

01 **Abstract**

02 **1. Introduccion**

03 **2. Contexto y trabajo relacionado**

04 **3. Arquitectura del sistema**

05 **4. Decision por utilidad y memoria**

06 **5. Baseline experimental reproducible**

07 **6. Trazabilidad radical y taxonomia de evidencia**

08 **7. Modelo conceptual y flujo fundador**

09 **8. Limitaciones**

10 **9. Trabajo futuro**

11 10. Conclusiones

12 Referencias

13 Anexo minimo: reproduccion y verificacion

Palabras clave: vida artificial, agentes autonomos, funcion de utilidad, simulacion 2D, memoria social, trazabilidad, reproducibilidad, comportamiento emergente.

Autor: Cosigein SL

Fecha: 2026-03-09

Repositorio: `artificial-word` — github.com/artificial-word

Version: 1.1 — Preprint tecnico / whitepaper fundacional

Abstract

Presentamos **Artificial World**, un sistema de simulacion multiagente orientado a civilizaciones emergentes, disenado alrededor de tres propiedades de primer orden: trazabilidad, reproducibilidad y honestidad tecnica. El proyecto combina un motor principal en Python con decision por utilidad, memoria espacial y social, persistencia SQLite y mecanismos de auditoria verificable, junto con una capa web separada que funciona como demo operativa con modelo conceptual compartido, pero sin integracion tecnica forzada con el motor principal.

Como baseline reproducible, documentamos una sesion canonica con semilla fija, 8 agentes y 200 ticks. En esta ejecucion se observan patrones consistentes de cohesion grupal bajo escasez, agotamiento zonal y divergencia individual exitosa. No presentamos esta sesion como validacion experimental exhaustiva, sino como una linea base verificable para futuras comparaciones controladas.

La contribucion principal del trabajo no es solo la simulacion en si, sino una metodologia de construccion y comunicacion para sistemas complejos: cada claim relevante debe clasificarse como `REAL`, `DEMO`, `PARCIAL` o `ROADMAP`, y debe poder rastrearse a codigo, tests, logs o artefactos observables. Defendemos que esta trazabilidad radical no es una caracteristica accesoria, sino una propiedad de diseno central para sistemas de IA y simulacion que aspiren a ser auditables.

Palabras clave: vida artificial, agentes autonomos, funcion de utilidad, simulacion 2D, memoria social, trazabilidad, reproducibilidad, comportamiento emergente.

1. Introduccion

Los sistemas de IA y simulacion suelen sufrir un problema recurrente: cuanto mas ambiciosa es su narrativa, mas dificil resulta distinguir entre lo que esta implementado, lo que funciona de forma parcial y lo que solo existe como vision futura. `Artificial World` parte de una premisa opuesta: un sistema complejo debe poder explicar sus decisiones, exponer sus limites y permitir verificacion externa.

En lugar de optimizar la presentacion por encima de la evidencia, el proyecto prioriza tres principios:

1. reproducibilidad por semilla fija
2. decision explicable mediante una funcion de utilidad publica

3. separacion explicita entre nucleo real, demo funcional y roadmap

El objetivo de este documento no es afirmar que el proyecto haya alcanzado una simulacion social completa, sino presentar una base tecnica defendible y una metodologia de trazabilidad aplicable a sistemas multiagente.

1.1 Alcance del documento

Este texto debe leerse como un **preprint tecnico** con baseline experimental reproducible, no como un paper cientifico ya endurecido por validacion estadistica amplia. Su proposito es:

- describir la arquitectura verificable del sistema
- fijar la frontera entre `REAL`, `DEMO`, `PARCIAL` y `ROADMAP`
- documentar una baseline reproducible inicial
- identificar los limites actuales sin sobreprometer

---

2. Contexto y trabajo relacionado

La tradicion de la vida artificial se mueve entre dos extremos. Por un lado, sistemas de reglas locales simples, como automatas celulares o `Boids`, logran comportamiento emergente a partir de reglas minimas. Por otro, sistemas con estado interno mas rico, como `Dwarf Fortress`, `Creatures` o `RimWorld`, aumentan la complejidad del agente pero suelen volver mas opaco el mecanismo de decision.

`Artificial World` se situa en la segunda familia, pero con una diferencia central: la seleccion de acciones se apoya en una funcion de utilidad explicita y reproducible. En lugar de depender de un mecanismo opaco, cada accion candidata se puntua a partir de modificadores observables ligados al estado fisiologico, a la memoria y a las relaciones sociales.

La arquitectura tambien toma distancia de los sistemas que mezclan producto, demo y vision futura en una misma superficie narrativa. Aqui se explicita una frontera metodologica: lo que existe y puede verificarse debe separarse de lo que solo se muestra como demo o se mantiene como roadmap.

---

3. Arquitectura del sistema

`Artificial World` se compone hoy de dos capas principales y un mecanismo transversal de auditoria:

Capa	Estado	Rol
Motor Python	`REAL`	Simulacion principal, persistencia, Modo Sombra, watchdog y cronica
Capa web	`DEMO` / `PARCIAL`	Flujo fundador, visualizacion y superficies de exploracion
Sistema Chess	`REAL`	Auditoria separada y lectura externa del repositorio

La separacion entre motor Python y capa web no se presenta como deuda oculta. Es una decision arquitectonica explicita: cada capa tiene un proposito distinto y su grado de realidad debe comunicarse sin ambiguedad.

3.1 Motor principal en Python

El motor Python es hoy la parte mas defendible del repositorio. Implementa:

- simulacion por ticks
- funcion de utilidad para seleccion de acciones
- memoria espacial y social
- persistencia SQLite

- Modo Sombra para evaluacion contrafactual
- watchdog para alertas estructuradas
- generacion de cronica verificable

El ciclo de simulacion por tick es:

```
actualizar_estado_interno()
percibir_entorno()
actualizar_memoria()
construir_contexto_decision()
puntuar_acciones()
ejecutar_accion_ganadora()
registrar_efectos()
```

3.2 Capa web separada

La capa web comparte el modelo conceptual del proyecto, pero no actua como interfaz directa del motor Python. Su funcion actual es servir como demo operativa y puerta de entrada al flujo fundador:

- seleccion de `CivilizationSeed`
- nombrado del heroe fundador
- nombrado del refugio fundador
- creacion de un mundo ligero con estado fundacional

Esa capa no debe presentarse como prueba de una unica base de verdad en tiempo real entre Python y JavaScript. La integracion total sigue en `ROADMAP`.

3.3 Sistema Chess

El `Sistema Chess` implementa una forma de auditoria separada: agentes dockerizados con acceso de solo lectura al repositorio revisan documentacion, backend, frontend, base de datos, tests y narrativa. Su valor dentro de este documento no es inflar la arquitectura, sino reforzar la tesis metodologica de verificabilidad externa.

---

4. Decision por utilidad y memoria

La puntuacion de una accion `a` para el agente `i` en el tick `t` sigue la forma general:

```
U(a, i, t) = U_base(a) + sum(delta_k) + epsilon(i, t)
```

donde:

- `U_base(a)` es la utilidad intrinseca de la accion
- `delta_k` agrupa los modificadores contextuales
- `epsilon(i, t)` es ruido determinista por semilla

Los modificadores principales son:

Modificador	Funcion
`hambre`	prioriza subsistencia cuando la carencia aumenta
`energia`	aumenta descanso en estados de fatiga
`riesgo`	favorece huida o evitacion en entornos hostiles
`rasgo`	ajusta preferencias segun arquetipo conductual
`anti_oscilacion`	penaliza retorno inmediato a la celda previa
`memoria`	bonifica movimiento hacia recursos visibles o recordados
`relaciones`	ajusta acciones sociales segun confianza, hostilidad y miedo
`directivas`	introduce instrucciones externas experimentales
`autonomia`	impone un override de supervivencia en estados criticos

Esta formulacion favorece explicabilidad y depuracion. La decision no depende de un componente opaco, sino de una puntuacion reconstruible a partir del estado del agente y del tick actual.

4.1 Memoria espacial y social

Cada agente mantiene dos registros persistentes entre ticks:

- **memoria espacial**: posiciones recientes de recursos y refugios
- **memoria social**: vectores de confianza, hostilidad y miedo hacia otros agentes

Esto permite que el sistema no reaccione solo al presente inmediato. Las decisiones futuras dependen de rastros del pasado, tanto en terminos de exploracion como de historia relacional.

4.2 Override de autonomia

Una caracteristica importante del motor es la capa de autonomia de emergencia. Cuando el agente entra en estados fisiologicos o de riesgo criticos, este mecanismo penaliza acciones incompatibles con la supervivencia y prioriza conductas de resguardo. Su funcion no es volver "inteligente" al sistema por decreto, sino imponer una restriccion de seguridad explicita sobre la decision.

---

5. Baseline experimental reproducible

Como punto de partida experimental, el proyecto documenta una sesion canonica con:

Parametro	Valor
Semilla fija	`42`
Mapa	`60 x 60`
Entidades iniciales	`8`
Ticks ejecutados	`200`
Fundador	`Tryndamere`
Refugio fundador	`Refugio Fundador`

5.1 Resultados agregados

Accion	Ejecuciones	Proporcion
`SEGUIR`	998	62.4%
`DESCANSAR`	492	30.8%
`RECOGER_RECURSO`	55	3.4%
`COMER`	55	3.4%
Total	**1600**	**100%**

Estado final resumido:

- supervivencia total: `8/8`
- hambre critica en `7/8` entidades
- alertas watchdog: `65`
- veredicto de sesion: `TENSION`

5.2 Observaciones iniciales

En esta baseline aparecen tres patrones relevantes:

1. **cohesion grupal bajo escasez**: la accion `SEGUIR` domina ampliamente la distribucion.
2. **agotamiento zonal**: varios agentes terminan concentrados en areas con recursos degradados.
3. **divergencia individual exitosa**: una entidad aislada finaliza con hambre sub-critica respecto al resto.

Estos resultados son interesantes porque surgen de la interaccion entre reglas explicitas y no de un guion hardcodeado para producir una narrativa concreta.

5.3 Reproducibilidad

La baseline puede regenerarse mediante:

```
python cronica_fundacional.py --seed 42 --ticks 200 --founder "Tryndamere" --refuge "Refugio Fundador"
```

El uso de semilla fija y ruido determinista permite reconstruir el mismo conjunto de decisiones a igualdad de estado inicial.

5.4 Interpretacion prudente

Esta sesion debe interpretarse como una **linea base reproducible**, no como validacion experimental exhaustiva. Su valor principal es metodologico:

- permite comparar versiones futuras del sistema
- deja artefactos verificables
- facilita detectar regresiones y cambios de comportamiento

No permite por si sola sostener afirmaciones generales fuertes sobre robustez estadistica o dinamica emergente universal.

6. Trazabilidad radical y taxonomia de evidencia

La trazabilidad radical, tal como se propone aqui, significa que para cualquier decision relevante debe existir una cadena de evidencia legible:

```
tick → estado → candidatos → puntuaciones → decision → efecto
```

Y para cualquier claim sobre el sistema:

```
claim → codigo / test / log / artefacto → verificacion externa
```

6.1 Taxonomia del estado de los componentes

Estado	Significado
`REAL`	Implementado, reproducible y verificable
`DEMO`	Funcional, pero no parte del nucleo principal
`PARCIAL`	Implementado de forma incompleta o con alcance limitado
`ROADMAP`	Disenado o deseado, pero no implementado

Aplicado al proyecto:

- el motor Python, la persistencia, el Modo Sombra y la cronica pertenecen a `REAL`
- la capa web entra en `DEMO` o `PARCIAL` segun el subsistema
- la integracion total Python/JS y la capa 3D permanecen en `ROADMAP`

Esta taxonomia no es un adorno editorial. Es parte central de la metodologia del proyecto: evitar que la ambicion narrativa sobrepase a la evidencia disponible.

---

7. Modelo conceptual y flujo fundador

El sistema propone un modelo conceptual compartido en torno a estas entidades:

- `World`
- `CivilizationSeed`
- `Refuge`
- `Hero`
- `Community`
- `MemoryEntry`
- `HistoricalRecord`
- `Territory`

El flujo fundador minimo defendible es:

```
semilla → heroe → refugio → mundo inicial → cronica fundacional
```

Este flujo no debe confundirse con una simulacion completa de civilizacion madura. Hoy representa una base verificable para mundos iniciales, no una implementacion plena de diplomacia, linajes o dinamicas comunitarias profundas.

7.1 Regla 2D / 3D

Una decision editorial y arquitectonica importante del proyecto es esta:

Capa	Rol	Estado
`2D`	verdad sistemica: mapa, recursos, refugios, rutas, influencia	implementada
`3D`	presencia futura: heroes encarnados, refugios visuales, eventos historicos	roadmap

La capa 3D no debe presentarse como funcionalidad existente. Su rol correcto es el de futura encarnacion visual subordinada a la verdad sistemica 2D.

---

8. Limitaciones

El trabajo tiene limites claros que deben declararse sin rodeos:

1. La baseline presentada usa una sola sesion pequena y no permite generalizacion fuerte.
2. Faltan multiples semillas, corridas mas largas y agregacion estadistica.
3. No se incluyen aun comparaciones sistematicas con baselines simples.
4. La capa web y otras superficies del repositorio pueden distraer si se confunden con el nucleo experimental.
5. `Community` existe hoy mas como contrato y estado inicial que como simulacion social rica.
6. La integracion Python/JS y la capa 3D siguen fuera del estado implementado.

---

9. Trabajo futuro

Las siguientes lineas reforzarian el valor cientifico y metodologico del sistema:

1. ejecutar multiples semillas y longitudes de corrida
2. comparar contra variantes sin memoria, sin relaciones o sin override de autonomia
3. medir supervivencia, dispersion, cohesion y consumo de recursos
4. enriquecer `Community`, `HistoricalRecord` y normas emergentes
5. mantener la taxonomia `REAL / DEMO / PARCIAL / ROADMAP` como contrato editorial permanente
6. decidir mas adelante si la integracion Python/JS justifica su coste

---

10. Conclusiones

`Artificial World` no debe defenderse hoy como un paper cientifico endurecido por evidencia extensa. Debe defenderse como un **preprint tecnico serio** que presenta una arquitectura reproducible de simulacion multiagente y una metodologia de trazabilidad radical.

Su valor principal actual no reside en la amplitud de sus resultados experimentales, sino en haber construido un sistema que obliga a distinguir entre evidencia, demo, parcialidad y vision futura. La baseline experimental con semilla fija no cierra la discusion cientifica, pero establece una base verificable sobre la que si puede construirse una validacion mas fuerte.

En ese sentido, la contribucion defendible del proyecto es doble:

1. una arquitectura simulada explicable y reproducible
2. una disciplina documental que obliga a que los claims apunten a evidencia

Esa combinacion ya constituye una aportacion tecnica y metodologica relevante para el diseno de sistemas complejos auditables.

Referencias

- Adams, T. (2006). *Dwarf Fortress*. Bay 12 Games.
- Bikhchandani, S., Hirshleifer, D., & Welch, I. (1992). A Theory of Fads, Fashion, Custom, and Cultural Change as Informational Cascades. *Journal of Political Economy*, 100(5), 992-1026.
- Conway, J. H. (1970). The Game of Life. *Scientific American*, 223(4), 4-10.
- Langton, C. G. (1989). Artificial Life. *Artificial Life*, SFI Studies in the Sciences of Complexity, 1-47.
- Rao, A. S., & Georgeff, M. P. (1991). Modeling Rational Agents within a BDI-Architecture. *KR*, 91, 473-484.
- Reynolds, C. W. (1987). Flocks, Herds, and Schools: A Distributed Behavioral Model. *SIGGRAPH Computer Graphics*, 21(4), 25-34.
- Sylvester, T. (2013). *RimWorld*. Ludeon Studios.
- Von Neumann, J., & Morgenstern, O. (1944). *Theory of Games and Economic Behavior*. Princeton University Press.

Anexo minimo: reproduccion y verificacion

Reproduccion de la baseline

```
pip install -r requirements.txt
$env:SDL_VIDEODRIVER="dummy"
$env:SDL_AUDIODRIVER="dummy"
python cronica_fundacional.py --seed 42 --ticks 200 --founder "Tryndamere" --refuge "Refugio Fundador"
```

Artefactos esperados

- `cronica_fundacional.json`
- `cronica_fundacional.md`
- `cronica_fundacional.pdf`

Taxonomia resumida

Componente	Estado
Motor Python 2D	`REAL`
Funcion de utilidad	`REAL`
Memoria espacial/social	`REAL`
Persistencia SQLite	`REAL`
Modo Sombra	`REAL`
Capa web	`DEMO`
HeroRefuge y flujo fundador web	`PARCIAL`
Integracion Python/JS	`ROADMAP`
Runtime 3D	`ROADMAP`

Artificial World · Documento definitivo premium generado desde `docs/PAPER_FINAL.md`